Hardware Implementation vs. CPU Implementation

Kaoutar Jahid, Victoria Hofmeister, Priyanka Ravinder, Mohul Sarkar

Overview

01 What's the Problem?

02 Objective

03 Background Concepts

04 Thesis/Hypothesis

05 Experimental Setup

06 Methodology

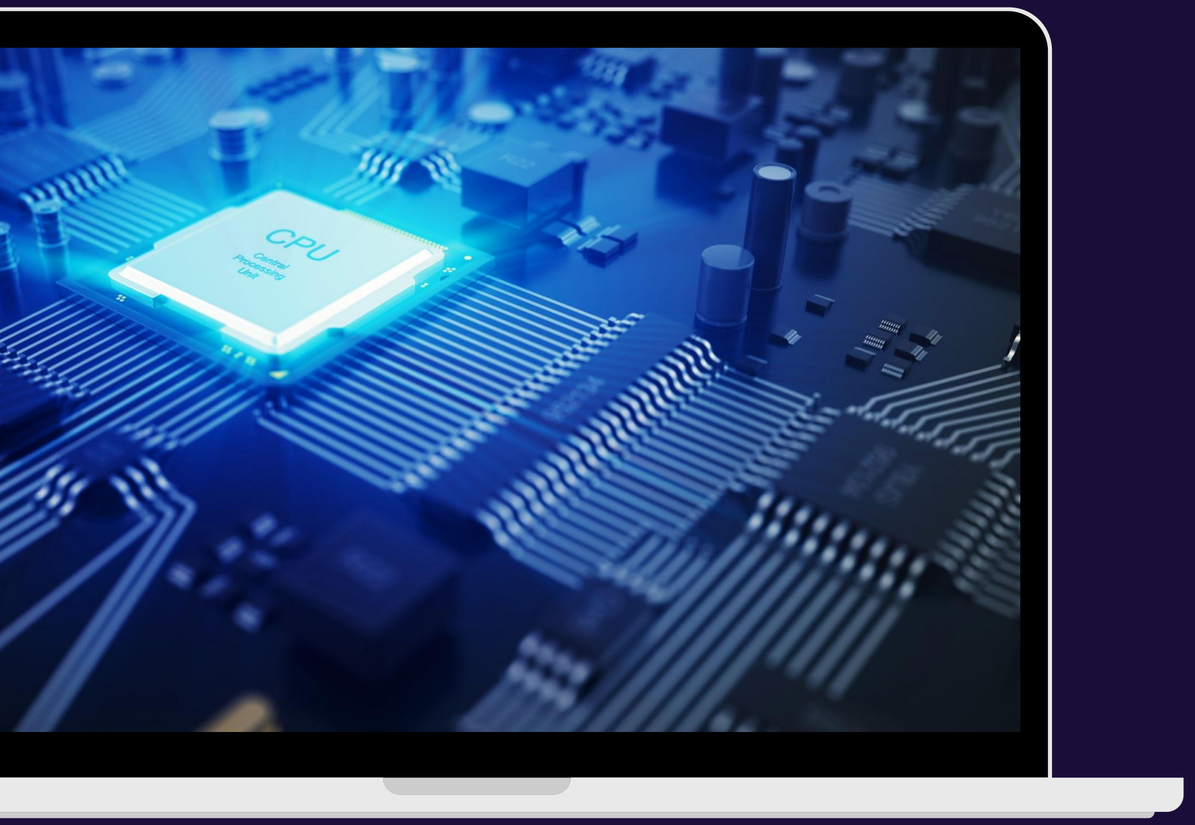
07 Metrics & Comparison

08 System Design

09 Results

10 Analysis/Discussion

What's The Problem?



- Modern systems balance **flexibility vs performance**
- CPUs are general-purpose and instruction-driven
- FPGAs enable **custom hardware specialization**

Core Question:

- How does:
  - **Instruction-driven execution (CPU)**
    compare to
  - **Custom datapath execution (FPGA)**

Goal

- Quantify performance differences for the same task
  Understand architectural tradeoffs

Note:

We're not just comparing devices—we're comparing *execution models*.

Objective

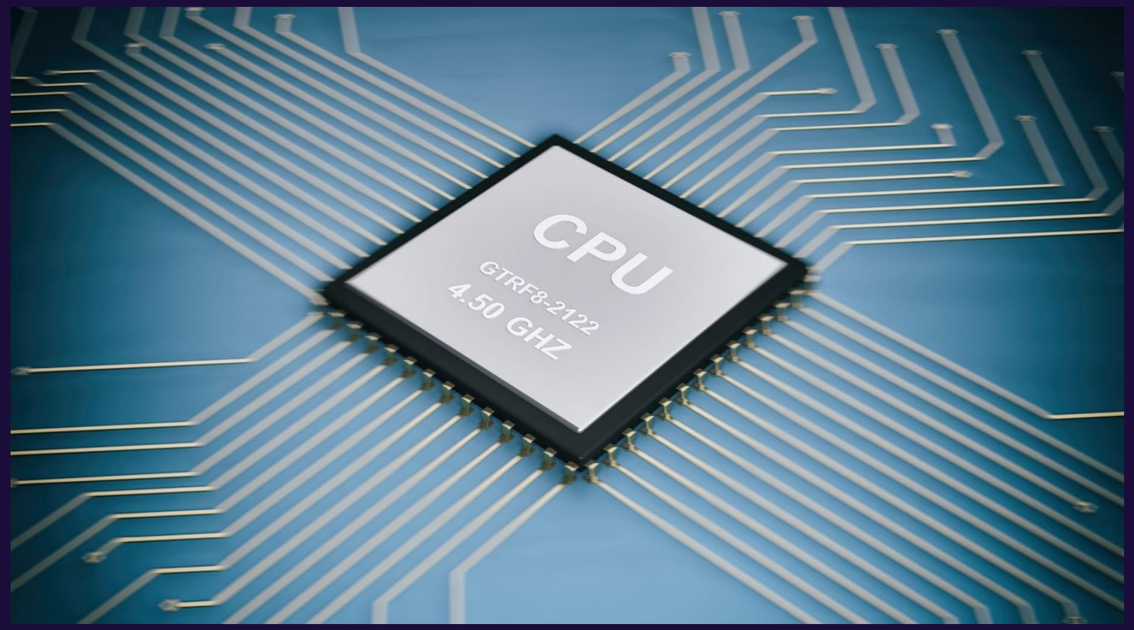
Benchmark CPU vs FPGA on a **fixed FIR workload**

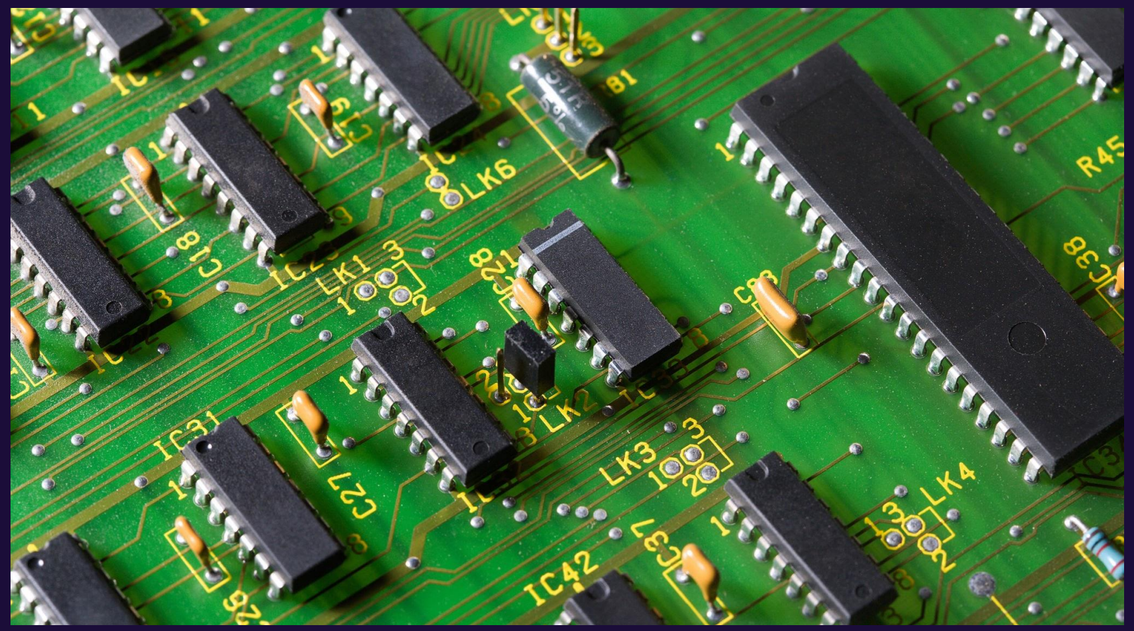
Focus:

- Execution organization, not hardware generation differences

What we want to measure:

- Performance
- Efficiency
- Scalability





Background Concepts

CPU (Temporal Execution)

One ALU reused repeatedly

Instruction-controlled

Sequential execution

FPGA (Spatial Execution)

Dedicated hardware blocks

Parallel computation

Execution defined by datapath

Thesis/Hypothesis

Architecture Tradeoff

- Reuse vs Replication

CPU	FPGA
Reuses hardware	Replicates hardware
Flexible	Specialized
Sequential	Parallel

Hypothesis

- FPGA achieves higher throughput via parallelism
- CPU provides flexibility at the cost of performance

Experimental Setup & Controlled Variables

Hardware

- x86 Laptop (CPU)
- Nexys A7 FPGA

Task

- FIR (Finite Impulse Response) Filter

To ensure fair comparison:

- Same input size
- Same coefficients
- Same numerical precision
- Same output requirements

Methodology

Approach

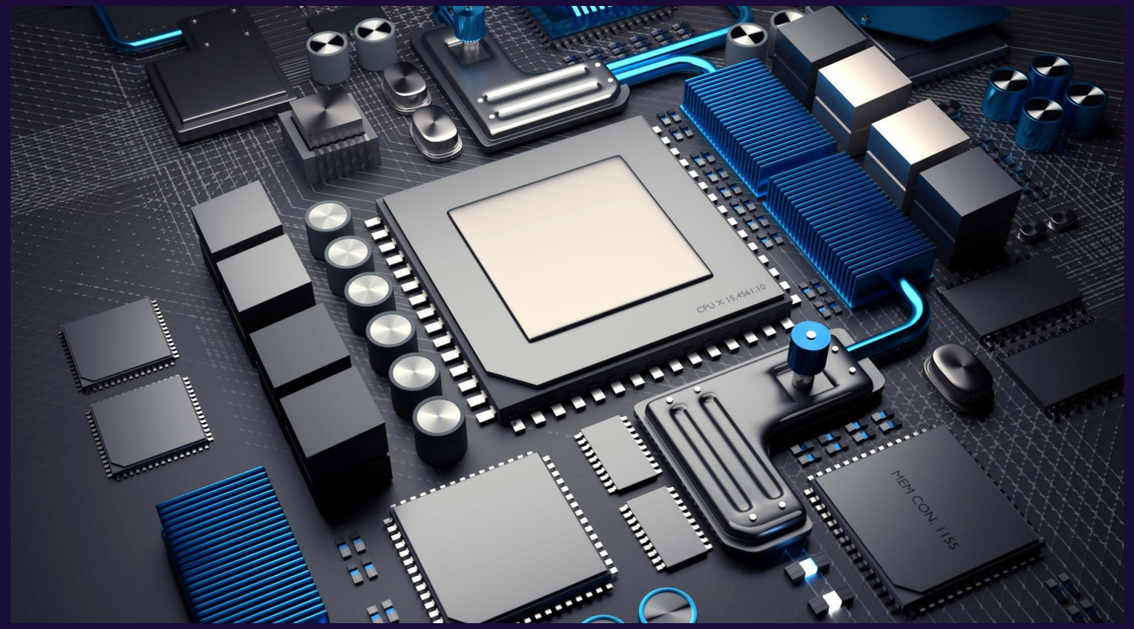
- Implement FIR filter on:
  - CPU (software)
  - FPGA (hardware)

Experiment Variation

- Vary number of taps

Measurement Strategy

- Measure execution time and efficiency metrics



Metrics & Comparison

CPU Metrics

Measured on CPU

- Execution time (wall-clock)
- Clock cycles
- CPI (cycles per instruction)
- Cycles per sample
- Throughput

FPGA Metrics

Measured on FPGA

- Clock frequency
- Cycles per sample
- Latency
- Throughput

Comparison

Key Metrics

- Throughput comparison
- Speedup:
  - FPGA vs CPU

Work per Clock

- CPU: Instructions per cycle
- FPGA: Outputs per clock

Results

Measured Performance

- Clock Frequency: 100 MHz
- Cycles per Sample: 9 cycles
- Latency: 90 ns
- Throughput: 11.11 MSamples/s

Efficiency Metrics

- Work per Clock: ~0.11 outputs/clock
- Deterministic Timing: Fixed cycle behavior

Resource Usage:

- LUTs: 63
- FFs: 95
- DSPs: 0
- BRAM: 0

Key Observations

- Dedicated hardware datapath (no instructions)
- Predictable execution (no variation)
- Sequential MAC design (1 multiplier reused)
- Very low resource usage
- Design meets timing with positive slack

Interpretation

High efficiency due to: no instruction overhead, direct hardware execution, Fixed 9 cycles/sample (very low), Lower throughput due to lower clock frequency (100 MHz)

Easily scalable: can increase frequency, can parallelize taps for higher performance

X86 CPU Results (FIR Benchmark)

Measured Performance

- Execution Time: 7.757 s
- CPU Time: 7.598 s
- Total Samples: 1,000,000,000
- Throughput: 132.06 MSamples/s

Efficiency Metrics

- Frequency: 3.5 GHz
- Total Cycles: 2.62×10^{10}
- Cycles per Sample: 26.2 cycles/sample

Key Observations

- 99% of time spent in FIR loop
- Single-threaded execution
- No SIMD (runs one value at a time)
- CPU is compute-bound

Elapsed Time: 7.757s

CPU Time:	7.598s
Total Thread Count:	1
Paused Time:	0s

Top Hotspots

This section lists the most active functions in your application. Optimizing th performance.

Function	Module	CPU Time	% of CPU Time
func@0x403bb0	fir.exe	7.530s	99.1%

Interpretation

- High throughput due to high clock speed and lower efficiency due to: instruction overhead, sequential execution and requires ~3× more cycles/sample than FPGA

Results

Metric	CPU (x86)	FPGA
Throughput	132.06 MS/s	11.11 MS/s
Cycles per Sample	26.2	9
Clock Frequency	3.5 GHz	100 MHz
Latency	Variable (non-deterministic)	90 ns (low)

CPU:

High throughput from high clock speed, Flexible (can change algorithm easily), Higher overhead (more cycles/sample)

FPGA:

Lower throughput due to lower frequency, Very efficient (fewer cycles/sample), Deterministic timing (predictable behavior)

Key Takeaway: CPU is faster in raw throughput, FPGA is more efficient per operation

Analysis/Discussion

Key Takeaways:

- Specialized hardware (FPGA) can significantly outperform CPUs for parallel tasks
- CPUs remain valuable for flexibility and general-purpose computing

Tradeoff Summary

- **CPU:**
  - Flexible, easy to program
  - Lower performance for parallel workloads
- **FPGA:**
  - High performance via specialization
  - Less flexible, higher design effort

Interpretation of Results

- **When FPGA outperforms CPU:**
  - High parallelism
  - Large workloads
- **When CPU is competitive:**
  - Small workloads
  - Low overhead tasks

Thank You!